

Нефункциональные требования

Архитектура ПО / Software Architecture

Non-Functional Requirements (*NFR*)

К концу вы сможете:

1. Объяснить, что такое **нефункциональные требования** (*non-functional requirements, NFR*) / **атрибуты качества** (*quality attributes*)
2. Сделать NFR **измеримым**: стимул → ответ → мера
3. Переписать плохие NFR на сценарии; видеть конфликты между качествами
4. Чётко контрастировать FR и NFR на одних и тех же продуктах

1. NFR: определение и карта качеств
2. Измеримость: сценарий качества
3. Много примеров good vs bad по атрибутам
4. FR vs NFR — бок о бок
5. Конфликты NFR и trade-offs
6. Compliance, availability и «не-цели»

NFR / атрибут качества

Ограничение или свойство **как** система выполняет функции: быстроедействие, надёжность, безопасность, удобство, сопровождаемость, масштаб, доступность, соответствие нормам. . .

Синонимы в литературе: quality attribute, quality requirement, *ility*-требования (scalability, reliability. . .).

Карта атрибутов (рабочий набор)

Атрибут	English
Производительность	performance
Надёжность	reliability
Доступность	availability
Безопасность	security
Удобство использования	usability
Сопровождаемость	maintainability
Масштабируемость	scalability
Соответствие нормам	compliance

Список не исчерпывающий (portability, observability...) — берите то, что критично продукту.

Почему «система должна быть быстрой» — плохой NFR

- Нет **контекста**: какая операция? при какой нагрузке?
- Нет **меры**: секунды? проценты? на чьём устройстве?
- Невозможно принять работу: вечный спор «уже быстро / ещё нет»

Запомните

Хороший NFR почти всегда выглядит как **сценарий** с числами или чётким наблюдаемым исходом.

Викторина

«p95 открытия ленты транзакций < 500 мс при 200 RPS» — это:

1. Функциональное требование
2. Нефункциональное (performance)
3. Только выбор СУБД
4. User story без роли

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

2. NFR про производительность с мерой и нагрузкой.

Сама возможность «открыть ленту» — отдельный FR.

Шаблон сценария качества

Элемент	Смысл
Источник (<i>source</i>)	кто/что инициирует
Стимул (<i>stimulus</i>)	событие (запрос, сбой, рост нагрузки)
Среда (<i>environment</i>)	норма / пик / авария
Артефакт	что затронуто (сервис, канал, данные)
Ответ (<i>response</i>)	что система делает
Мера (<i>response measure</i>)	как измерить успех

Для курса достаточно: **когда + условия + ответ + мера.**

Пример сценария — performance (магазин)

Когда покупатель открывает карточку товара

при каталоге 100k SKU и фоне 500 просмотров/с

система отдаёт карточку (HTML/JSON)

мера: p95 latency < 300 мс на стороне API; ошибка 5xx < 0,1%.

Пример — reliability / availability (банк)

Когда падает один экземпляр сервиса переводов
в рабочее время

система продолжает принимать переводы на оставшихся экземплярах

мера: RTO восстановления сервиса ≤ 2 мин; ни один принятый перевод не дублируется и не «теряется в воздухе».

Availability часто выражают процентом аптайма (например 99,9%) — поясняйте, что именно измеряете.

Пример — security (портал)

Когда студент пытается открыть чужую ведомость по прямому URL/ID

система отказывает в доступе

мера: HTTP 403; в теле нет чужих PII; событие в audit log за < 5 с.

Пример — usability

Когда новый студент впервые сдаёт домашку с телефона
при обычном 4G

мера: ≥ 8 из 10 в usability-тесте завершают сдачу за ≤ 3 мин без подсказки преподавателя; не более 1 критической ошибки UI на сессию.

Usability тоже можно делать проверяемой — не только «удобно».

Пример — maintainability

Когда команда добавляет новый канал уведомлений (push)

при уже существующих email/SMS

мера: изменение локализовано в модуле Notify; типовая задача ≤ 3 человеко-дня у разработчика, знакомого с кодовой базой; без правок модуля Payments.

Пример — scalability

Когда в неделю экзаменов нагрузка на портал растёт $\times 10$ к обычному вторнику

система сохраняет p95 входа < 2 с

мера: достигается горизонтальным масштабированием app-tier без переписывания бизнес-логики сдачи файлов.

Викторина

Чего не хватает: «API должен отвечать быстро при высокой нагрузке»?

1. Только названия языка программирования
2. Конкретной операции, профиля нагрузки и численной меры
3. Обязательного упоминания микросервисов
4. Списка кнопок UI

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

2. Без операции, нагрузки и меры NFR нельзя проверить.

Performance — до / после

Плохо

Система быстрая

Оптимизировать всё

Лучше

Поиск по каталогу: $p_{95} < 400$ мс при 1k запросов/с

Checkout step «pay»: $p_{99} < 1,5$ с исключая 3DS банка

Security — до / после

Плохо	Лучше
Данные защищены	Пароли только как salted hash; offline-bruteforce стойкость по политике
Есть безопасность	Админские действия — MFA; сессия 15 мин простоя
GDPR-ready (без деталей)	Право на удаление аккаунта: PII стирается ≤ 30 дней, отчёт в тикете

Reliability / availability — до / после

Плохо	Лучше
Не должна падать	Успешные платежи идемпотентны по Idempotency-Key
High availability	99,9% месячного аптайма API оплаты (исключая плановые окна)
Есть бэкапы	RPO $\leq$ 15 мин; проверка restore раз в квартал с протоколом

Maintainability / scalability — до / после

Плохо

Код чистый

Масштабируемая

Лучше

Новый отчёт по заказам без изменения модуля Auth

Пик Black Friday ×20: читающий каталог масштабируется отдельно от checkout

- Хранение логов доступа не менее N месяцев (регулятор)
- Запрет хранения полного PAN карты — только токен провайдера (PCI-DSS идея)
- Согласие на cookies / аналитику до включения трекеров

Compliance часто выглядит как NFR с юридической мерой «соответствует / не соответствует».

Один продукт — две колонки (чат)

FR

Отправить текст в диалог

Статусы delivered / read

Прикрепить файл ≤ 25 МБ

NFR

Доставка online p95 < 1 с

История 1М сообщений доступна < 2 с на первые 50

Загрузка файла устойчива к обрыву (resume)

Банк — бок о бок

FR

Создать перевод

Заблокировать карту

Показать выписку PDF

NFR

Авторизация перевода < 3 с end-to-end

Блокировка доходит до процессинга ≤ 60 с

Генерация PDF ≤ 5 с для месяца с ≤ 500 операций

Кампусный портал — бок о бок

FR

Загрузить домашку до дедлайна
Преподаватель скачивает архив
Студент видит только свои оценки

NFR

Пик 8:55: p95 upload init < 2 с
Архив 200 файлов $\leq$ 1 мин
Попытка IDOR $\rightarrow$ 403 + audit

IoT — бок о бок

FR

Включить полив по порогу

Локальный режим offline

Журнал телеметрии

NFR

Команда до клапана ≤ 2 с при нормальной сети

При обрыве облака локальная логика продолжает цикл

Хранение 90 дней; запрос суток < 3 с

«При потере связи контроллер выполняет последнее расписание» —

- Можно считать **FR** (функция деградации)
- Рядом часто стоит **NFR**: время до перехода в offline-режим ≤ 10 с

Частая ошибка

Спорить ярлыками вместо того, чтобы сделать поведение и меру явными.

Викторина

«Отмена заказа до отгрузки возвращает деньги на карту» — в первую очередь:

1. FR (поведение возврата)
2. Только scalability
3. Выбор платёжного SDK
4. Non-goal

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

1. Это функциональное поведение.

NFR рядом мог бы быть: «возврат отражается в истории ≤ 24 ч (зависит от банка)».

Нельзя максимизировать всё:

- Security vs usability (MFA vs «вход в один тап»)
- Performance vs cost (больше реплик — дороже)
- Consistency vs availability/latency (распределённые данные)
- Maintainability vs time-to-market (идеальные границы vs дедлайн)

Архитектура — явный **компромисс** (*trade-off*) и приоритеты.

Мини-кейс: конфликт на портале

Заказчик хочет:

1. Вход без пароля по «волшебной ссылке» навсегда (usability)
2. Никаких рисков чужих оценок (security)

Возможный компромисс (пример, не догма):

- Magic link на 15 мин + устройство trust на 30 дней
- Критичные действия (смена email) — повторная MFA

Таблица trade-off (шаблон)

Выбор		Выигрыш	Цена / риск
Строгий	MFA	security	usability, поддержка
всегда			
Кэш	витрины	performance	устаревшие цены
агрессивно			
Один	монолит	time-to-market	maintainability позже
на старте			

Викторина

Какая пара — классический конфликт NFR?

1. «Создать заказ» и «отменить заказ» (оба FR)
2. Максимальная security и максимальная usability входа
3. Название таблицы БД и цвет логотипа
4. NFR никогда не конфликтуют

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

2. Security и usability часто тянут в разные стороны — нужен явный trade-off.

Викторина

Верно или неверно?

Для учебного MVP нормально явно записать non-goal: «пока не целимся в 99,99% availability как у глобального банка».

1. Верно
2. Неверно — всегда нужны банковские SLA

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

Верно. Явные не-цели (*non-goals*) берегут бюджет и честность приёмки.

Как NFR двигают архитектуру

Доминирующий NFR	Частые следствия
Performance / scale	кэш, CQRS-идеи, CDN, шардирование
Security	границы доверия, IdP, сегментация данных
Availability	реплики, health checks, деградация
Maintainability	модульный монолит, чистые API

Сначала приоритеты NFR — потом стиль (клиент–сервер, слои, модули).

Чеклист качества формулировки NFR

1. Есть сценарий (когда / при каких условиях)?
2. Есть численная или однозначная мера?
3. Понятно, что измеряем (API, UI, канал)?
4. Есть приоритет относительно других NFR?
5. Зафиксированы non-goals?

1. NFR описывают *качество* выполнения функций
2. Делайте NFR сценариями с мерой — иначе это лозунги
3. FR и NFR дополняют друг друга на одном продукте
4. NFR конфликтуют — архитектура выбирает trade-offs
5. Non-goals так же важны, как амбициозные SLA

нефункциональное требование (*NFR*) · атрибут качества (*quality attribute*)
сценарий (*scenario*) · стимул · ответ · мера
performance · reliability · availability · security
usability · maintainability · scalability · compliance
trade-off · non-goals

Практика

04-fr-nfr-practice.md: сортировка FR/NFR, переписывание, сценарии, конфликтная пара

Связь с планом курса: усиливает неделю 2 (атрибуты качества) и питает архитектурные решения недель 3+.

Что ещё мутно: сценарии, меры, конфликты или граница с FR?